

-  GUÍA DE INTEGRACIÓN — ANDREA
 -  ARCHIVOS QUE SON TU RESPONSABILIDAD
 -  ARQUITECTURA DEL GATEWAY (Lo que controlas)
 -  PRE-REQUISITOS
 -  PASO A PASO COMPLETO
 - PASO 1 — Clonar el repositorio (solo si no lo tienes)
 - PASO 2 — Configurar identidad Git
 - PASO 3 — Asegurarte de estar en develop actualizada
 - PASO 4 — Crear tu rama de trabajo
 - PASO 5 — Compilar ambos servicios para verificar
 - PASO 6 — Agregar SOLO tus carpetas al staging
 - PASO 7 — Revisar los cambios antes de commitear
 - PASO 8 — Hacer commit
 - PASO 9 — Subir tu rama a GitHub
 - PASO 10 — Verificar en GitHub
 - PASO 11 — Crear el Pull Request
 -  ACTUALIZAR TU RAMA CON CAMBIOS DE DEVELOP
 -  PROBLEMAS COMUNES Y SOLUCIONES
 - El gateway no compila — error de Spring Cloud
 - Error de CORS en producción
 - "Port 8080 already in use"
 - El user-service no conecta a la BD
 -  CHECKLIST PERSONAL — ANDREA
 - Preparación
 - Antes del commit
 - PR
 -  INFORMACIÓN TÉCNICA



GUÍA DE INTEGRACIÓN — ANDREA

Componentes a cargo: `user-service/` + `gateway/` **Rama de trabajo:** `feature/andrea-user-gateway` **Ejecutar en:** FASE 3 (después de que Rachel haya mergeado su PR)

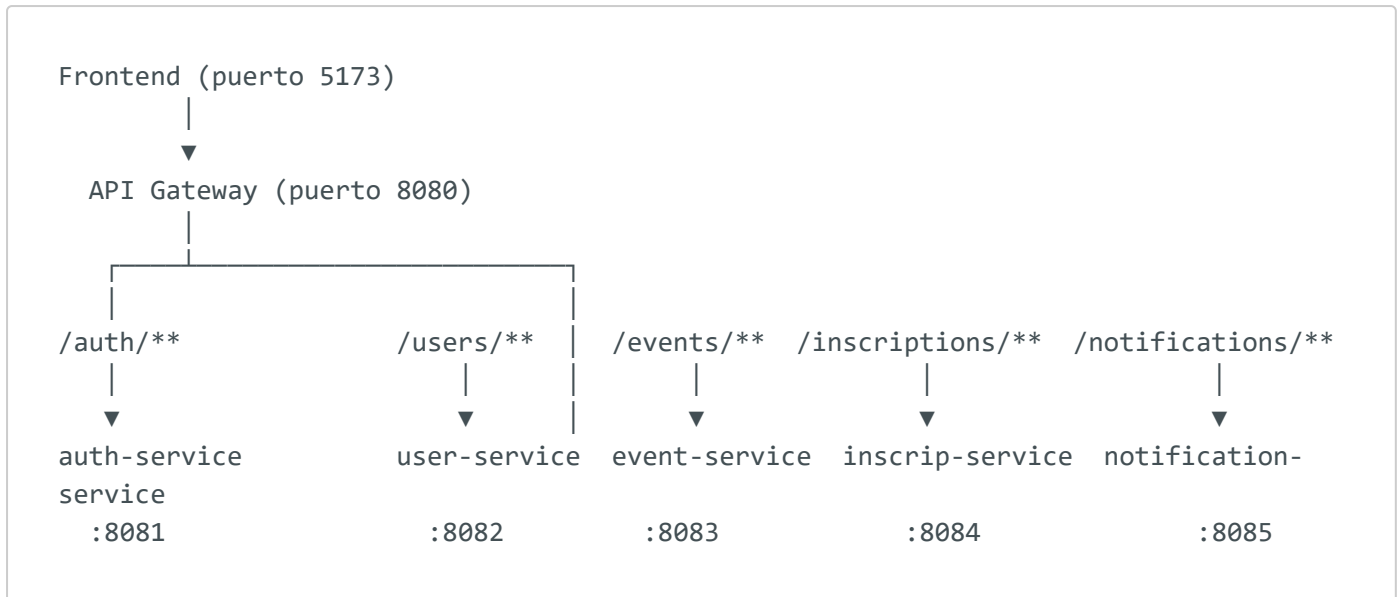
ARCHIVOS QUE SON TU RESPONSABILIDAD

```
user-service/
├── src/
│   ├── main/
│   │   ├── java/com/eventos/user_service/
│   │   │   ├── UserServiceApplication.java
│   │   │   ├── controller/
│   │   │   │   ├── UserProfileController.java
│   │   │   │   └── HealthController.java
│   │   │   ├── dto/
│   │   │   │   ├── UserProfileRequest.java
│   │   │   │   └── UserProfileResponse.java
│   │   │   ├── entity/
│   │   │   │   └── UserProfile.java
│   │   │   ├── repository/
│   │   │   │   └── UserProfileRepository.java
│   │   │   └── service/
│   │   │       └── UserProfileService.java
│   │   └── resources/
│   │       └── application.properties
│   └── test/
├── .mvn/
├── pom.xml
└── mvnw / mvnw.cmd
```

```
gateway/
├── src/
│   ├── main/
│   │   ├── java/com/eventos/api_gateway/
│   │   │   ├── ApiGatewayApplication.java
│   │   │   ├── config/
│   │   │   │   └── CorsConfig.java
│   │   │   ├── controller/
│   │   │   │   └── HealthController.java
│   │   └── resources/
│   │       └── application.properties
│   └── test/
├── .mvn/
├── pom.xml
└── mvnw / mvnw.cmd
```

 El **gateway** es el componente más crítico del sistema: **todas las peticiones del frontend pasan por aquí**. Es el enrutador central que redirige al servicio correcto.

ARQUITECTURA DEL GATEWAY (Lo que controlas)



Tu gateway enruta:

- `/auth/**` → `http://localhost:8081` (Rachel)
- `/users/**` → `http://localhost:8082` (tú)
- `/events/**` → `http://localhost:8083` (Braylin)
- `/inscripciones/**` → `http://localhost:8084` (Eliana)
- `/notifications/**` → `http://localhost:8085` (Rachel)



PRE-REQUISITOS

```
# Verificar Git
git --version

# Verificar Java 17
java -version
# Debe ser: openjdk version "17.x.x"

# Verificar que tienes acceso al repo
git remote -v
```



PASO A PASO COMPLETO

PASO 1 — Clonar el repositorio (solo si no lo tienes)

```
git clone [URL_DEL_REPO]
cd Eventos-Microservicios
```

PASO 2 — Configurar identidad Git

```
git config user.name "Andrea"
git config user.email "tu-email@gmail.com"
```

PASO 3 — Asegurarte de estar en develop actualizada

⚠️ **ESPERA** a que Rachel confirme que su PR (Fase 2) fue mergeado antes de ejecutar este paso.

```
git checkout develop
git pull origin develop
```

Verifica que ves el commit de Rachel:

```
git log --oneline -5
# Debe aparecer: feat(auth): auth-service y notification-service completos
```

PASO 4 — Crear tu rama de trabajo

```
git checkout -b feature/andrea-user-gateway
```

Confirma:

```
git branch  
# * feature/andrea-user-gateway
```

PASO 5 — Compilar ambos servicios para verificar

```
# Compilar user-service  
cd user-service  
./mvnw package -DskipTests  
cd ..  
  
# Compilar gateway  
cd gateway  
./mvnw package -DskipTests  
cd ..
```

Ambos deben mostrar **[INFO] BUILD SUCCESS**.

PASO 6 — Agregar SOLO tus carpetas al staging

 **SOLO** agrega **user-service/** y **gateway/**. Nada más.

```
# Agregar user-service  
git add user-service/  
  
# Agregar gateway  
git add gateway/  
  
# VERIFICAR inmediatamente  
git status
```

Salida esperada:

On branch feature/andrea-user-gateway

Changes to be committed:

```
modified: gateway/src/main/java/com/eventos/api_gateway/...
modified: gateway/src/main/resources/application.properties
modified: user-service/src/main/java/com/eventos/user_service/...
modified: user-service/src/main/resources/application.properties
```

NO debe aparecer: auth-service/, event-service/, inscrip-service/, frontend-v2/

Si aparece algo que no es tuyo:

```
git restore --staged auth-service/
git restore --staged event-service/
git restore --staged inscrip-service/
git restore --staged frontend-v2/
git restore --staged notification-service/
```

PASO 7 — Revisar los cambios antes de commit

```
# Ver qué cambiaste en application.properties del gateway
git diff --staged gateway/src/main/resources/application.properties

# Ver qué cambiaste en user-service
git diff --staged user-service/
```

PASO 8 — Hacer commit

```
git commit -m "feat(gateway+user): API Gateway con rutas completas y user-service"
```

gateway:

- Rutas configuradas para todos los microservicios (auth, user, event, inscrip, notifications)
- CORS configurado (CorsConfig.java)
- Puerto: 8080

user-service:

- Gestión de perfiles de usuario
- UserProfileController, UserProfileService
- Entidad UserProfile con JPA
- application.properties → Railway MySQL (user_db)"

PASO 9 — Subir tu rama a GitHub

Primera vez:

```
git push -u origin feature/andrea-user-gateway
```

Cambios adicionales:

```
git push origin feature/andrea-user-gateway
```

Force push (cuando sea necesario):

```
git push origin feature/andrea-user-gateway --force-with-lease
```

PASO 10 — Verificar en GitHub

1. Abre GitHub → tu repositorio
2. Selecciona rama `feature/andrea-user-gateway`
3. Verifica árbol: solo deben aparecer `gateway/` y `user-service/` como modificados
4. Revisa los archivos críticos:
 - `gateway/src/main/resources/application.properties` — Rutas completas
 - `gateway/src/main/java/com/eventos/api_gateway/config/CorsConfig.java` — CORS
 - `user-service/src/main/java/com/eventos/user_service/controller/UserProfileController.java`

PASO 11 — Crear el Pull Request

Por GitHub UI:

1. Clic en "Compare & pull request"

2. Configura:

- **Base:** `develop` ← ¡importante!
- **Compare:** `feature/andrea-user-gateway`
- **Título:** `feat(gateway+user): API Gateway y user-service completos`
- **Descripción:**

```
## API Gateway
- Enruta `/auth/**` → auth-service:8081
- Enruta `/users/**` → user-service:8082
- Enruta `/events/**` → event-service:8083
- Enruta `/inscriptions/**` → inscrip-service:8084
- Enruta `/notifications/**` → notification-service:8085
- CORS configurado para frontend en localhost:5173

## User Service
- CRUD de perfiles de usuario
- Conectado a user_db en Railway MySQL

## Pruebas realizadas
- [ ] Gateway compila
- [ ] User-service compila
- [ ] GET /users/health responde 200
- [ ] Las rutas del gateway están correctas
```

3. Clic "Create Pull Request"

Por CLI:

```
gh pr create \
  --base develop \
  --head feature/andrea-user-gateway \
  --title "feat(gateway+user): API Gateway y user-service completos" \
  --body "Gateway con rutas completas a todos los microservicios + user-service
para gestión de perfiles"
```

ACTUALIZAR TU RAMA CON CAMBIOS DE DEVELOP

```
git checkout develop
git pull origin develop
```



```
git checkout feature/andrea-user-gateway
git merge develop
# Resolver conflictos si los hay (no debería haber)
git push origin feature/andrea-user-gateway
```

PROBLEMAS COMUNES Y SOLUCIONES

El gateway no compila — error de Spring Cloud

```
cd gateway
./mvnw dependency:tree | grep cloud
# Verifica que spring-cloud-gateway-server está en el pom.xml
```

Error de CORS en producción

El gateway ya tiene `CorsConfig.java`. Si hay errores de CORS:

```
# Verificar la configuración en:
cat gateway/src/main/java/com/eventos/api_gateway/config/CorsConfig.java
```

Debe tener el origen del frontend configurado.

"Port 8080 already in use"

```
# En Windows
netstat -ano | findstr :8080
taskkill /PID [PID_NÚMERO] /F
```

El user-service no conecta a la BD

```
# Verificar application.properties
cat user-service/src/main/resources/application.properties
# El MYSQL_URL debe apuntar a jdbc:mysql://localhost:3306/user_db
# O a la URL de Railway si estás en producción
```



CHECKLIST PERSONAL — ANDREA

Preparación

- ☐ Esperé confirmación de Rachel (Fase 2 mergeada)
- ☐ `git checkout develop && git pull origin develop` ejecutado
- ☐ Vi el commit de Rachel en `git log`
- ☐ Creé rama `feature/andrea-user-gateway`

Antes del commit

- ☐ `user-service/` compila: `./mvnw package -DskipTests`
- ☐ `gateway/` compila: `./mvnw package -DskipTests`
- ☐ Solo `user-service/` y `gateway/` en staging (`git status` revisado)
- ☐ Mensaje de commit descriptivo

PR

- ☐ Push exitoso a `feature/andrea-user-gateway`
- ☐ Verificado en GitHub: solo cambios en `user-service/` y `gateway/`
- ☐ **Base:** `develop` ✓
- ☐ PR creado y Rachel notificada



INFORMACIÓN TÉCNICA

gateway:

- Puerto: `8080`

- Tecnología: Spring Cloud Gateway Server WebMVC
- CorsConfig permite origen: <http://localhost:5173> y Railway frontend

user-service:

- Puerto: [8082](#)
- Base de datos: [user_db](#) (Railway MySQL)

Endpoints de user-service:

GET	/users/{userId}	→ Obtener perfil
POST	/users	→ Crear perfil
PUT	/users/{userId}	→ Actualizar perfil
DELETE	/users/{userId}	→ Eliminar perfil
GET	/users/health	→ Health check

Verificar gateway funcionando:

```
# Si todos los servicios están corriendo, el gateway debe enrutar:  
curl http://localhost:8080/auth/health # → auth-service  
curl http://localhost:8080/users/health # → user-service (tú)  
curl http://localhost:8080/events/health # → event-service
```